

DOKUMEN FLOW SISTEM

NetMonitor API

Penjelasan alur kerja backend monitoring jaringan berbasis Fiber, GORM, dan MySQL: mulai dari startup aplikasi, autentikasi, otorisasi role, CRUD resource, data monitoring, alert, notifikasi, topologi, sampai data ML observability.

Project	be-project-smt6-polteksci
Stack	Go, Fiber, GORM, MySQL, Docker
Base URL	http://localhost:8080
Tanggal dokumen	29 Mei 2026
Sumber utama	routes/api.go , app/models/models.go , controller, middleware, config, database, Dockerfile



Daftar Isi

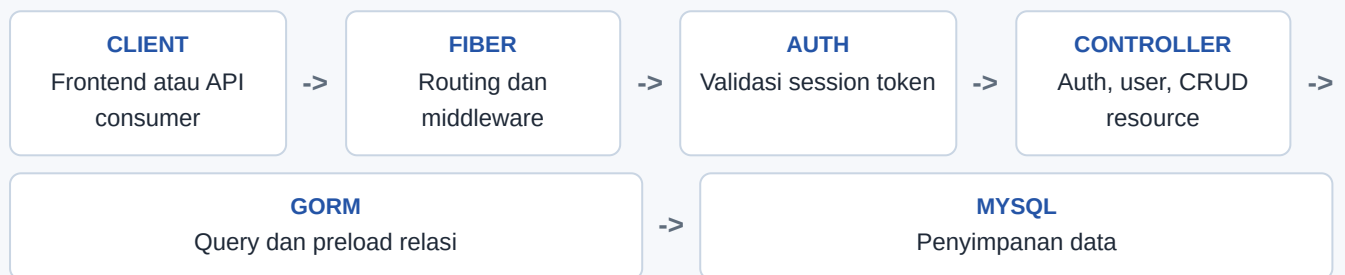
1. Ringkasan Sistem
2. Arsitektur dan Komponen
3. Flow Startup Aplikasi
4. Flow Request API
5. Flow Autentikasi
6. Flow Otorisasi Role
7. Flow CRUD Resource
8. Flow Domain Monitoring
9. Flow Database dan Relasi Data
10. Daftar Endpoint
11. Flow Deployment
12. Error Handling dan Response
13. Catatan Implementasi

1. Ringkasan Sistem

NetMonitor API adalah backend untuk sistem monitoring jaringan. Sistem menyimpan data user, session login, inventory device, status device, konfigurasi monitoring, alert, notifikasi, activity log, relasi topologi jaringan, prediksi ML, dan anomali ML.

API menggunakan pola REST. Endpoint publik hanya root, health check, dan login. Endpoint lain berada di bawah `/api` dan dilindungi middleware autentikasi berbasis session token. Token dikirim melalui header `Authorization: Bearer <token>` atau query `access_token`.

Gambaran Besar Alur Sistem



Tujuan Fungsional

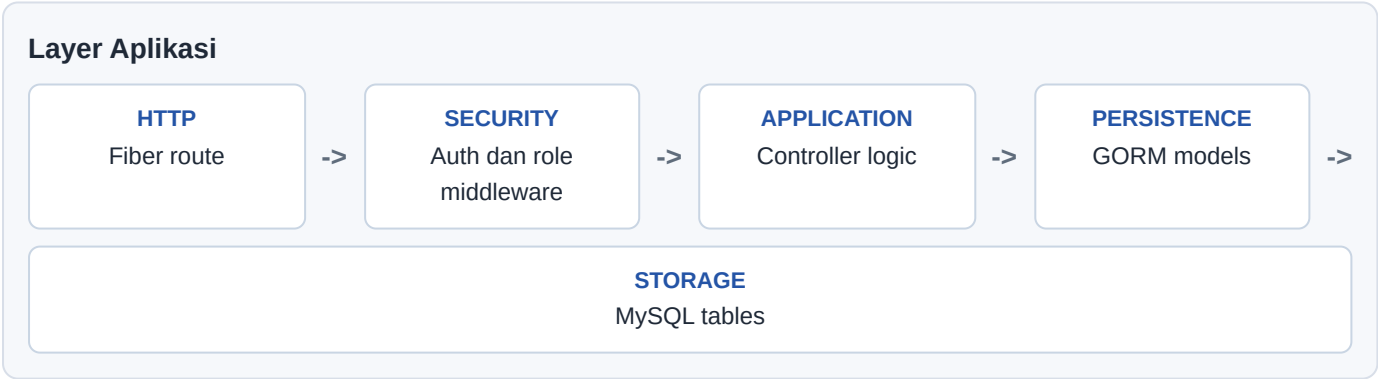
- Mengelola user dan hak akses.
- Mengelola device jaringan dan layanan.
- Menyimpan status performa device.
- Menyimpan konfigurasi monitoring ping dan TCP.
- Mengelola alert dan notifikasi.
- Menyimpan topologi link antar-device.
- Menyimpan output prediksi dan anomali ML.

Karakter Sistem Saat Ini

- Backend API, bukan worker monitoring aktif.
- CRUD generik untuk mayoritas resource.
- Session token disimpan di tabel `sessions`.
- Database dibuat otomatis jika belum ada.
- Schema dibuat dan disesuaikan melalui GORM `AutoMigrate`.

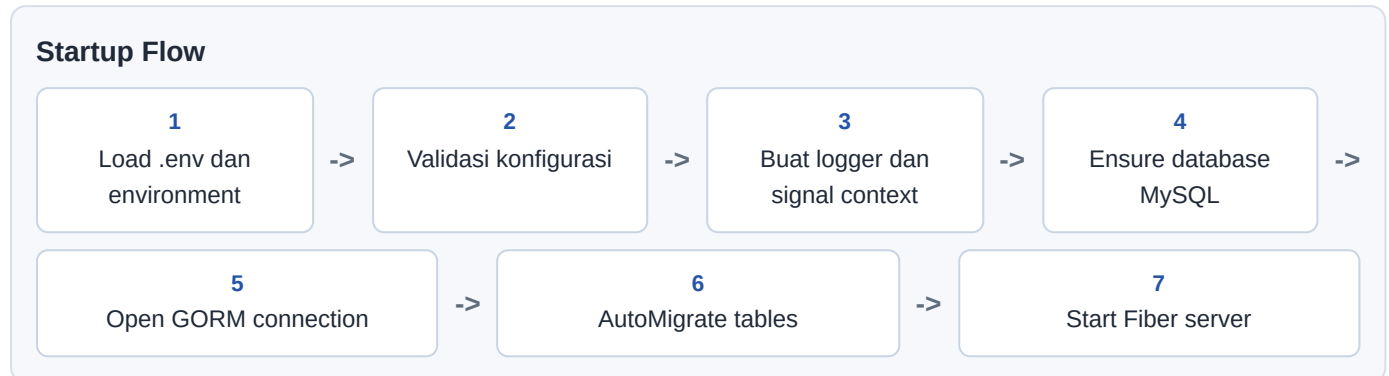
2. Arsitektur dan Komponen

Komponen	Lokasi	Tanggung Jawab
Entry point	<code>main.go</code> , <code>cmd/server/main.go</code>	Load konfigurasi, membuat logger, membuat context shutdown, menjalankan bootstrap aplikasi.
Config	<code>config/app.go</code>	Membaca <code>.env</code> , mengatur app name, port, CORS, TTL session, dan koneksi MySQL.
Logger	<code>config/logger.go</code>	Mencetak log startup, error, dan shutdown dengan format ringkas.
Database	<code>database/database.go</code>	Membuat database jika belum ada, membuka koneksi GORM, ping MySQL, dan AutoMigrate model.
Bootstrap	<code>bootstrap/app.go</code>	Membuat Fiber app, memasang recovery dan CORS, register route, listen server, graceful shutdown.
Routes	<code>routes/api.go</code>	Mendefinisikan endpoint publik, endpoint auth, protected group, dan resource CRUD.
Middleware	<code>app/http/middleware/auth.go</code>	Validasi session token dan pengecekan role.
Controllers	<code>app/http/controllers</code>	Menangani login, logout, profil user, CRUD user, dan CRUD resource generik.
Models	<code>app/models/models.go</code>	Definisi entity, enum, relasi, nama tabel khusus, dan daftar model untuk migrasi.



3. Flow Startup Aplikasi

Saat aplikasi dijalankan dengan `go run .`, binary Docker, atau `go run ./cmd/server`, sistem menjalankan flow startup berikut.



1. `config.Load()` membaca `.env` memakai `godotenv`, lalu fallback ke environment/default value.
2. Jika `APP_ENV=production`, nilai `JWT_SECRET` tidak boleh tetap default. Catatan: flow auth saat ini memakai session token database, bukan JWT payload.
3. `database.Connect()` membuka koneksi ke server MySQL tanpa memilih database, lalu menjalankan `CREATE DATABASE IF NOT EXISTS`.
4. GORM membuka koneksi ke database target, mengatur connection pool, melakukan ping, lalu menjalankan `AutoMigrate` untuk semua model.
5. Fiber app dibuat dengan timeout baca 10 detik, middleware recover, dan CORS.
6. Route didaftarkan melalui `routes.Register()`.
7. Server listen pada `:PORT`, default `:8080`.
8. Saat menerima `SIGINT` atau `SIGTERM`, aplikasi shutdown secara graceful dengan timeout 10 detik.

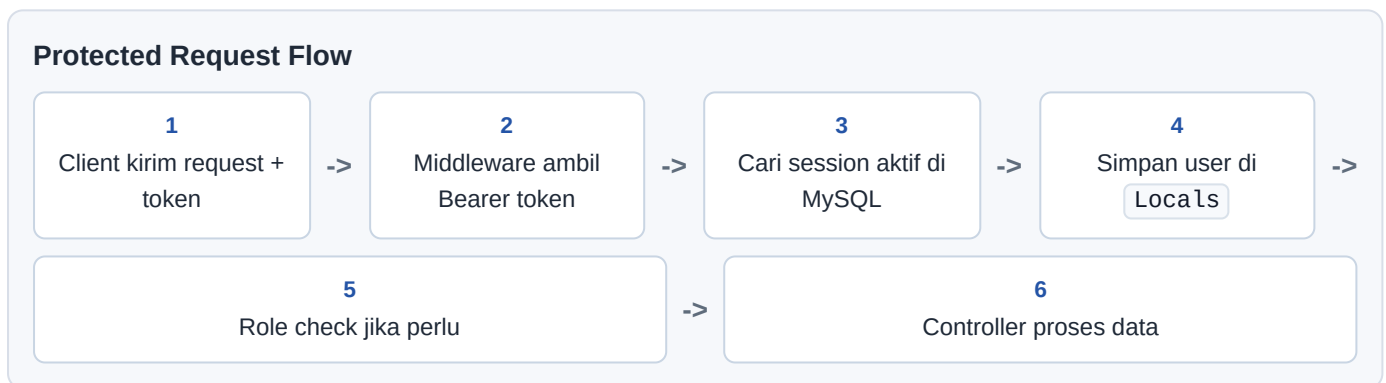
4. Flow Request API

Semua request masuk ke Fiber. Endpoint publik dapat langsung merespons, sedangkan endpoint protected melewati middleware autentikasi lebih dulu.

Endpoint Publik

Method	Path	Flow
GET	/	Mengembalikan nama aplikasi, deskripsi, status running, dan daftar endpoint utama.
GET	/healthz	Mengembalikan status <code>ok</code> , stack aplikasi, dan konfigurasi host/port/database MySQL.
POST	/api/auth/login, /api/login	Login user dan membuat session token.

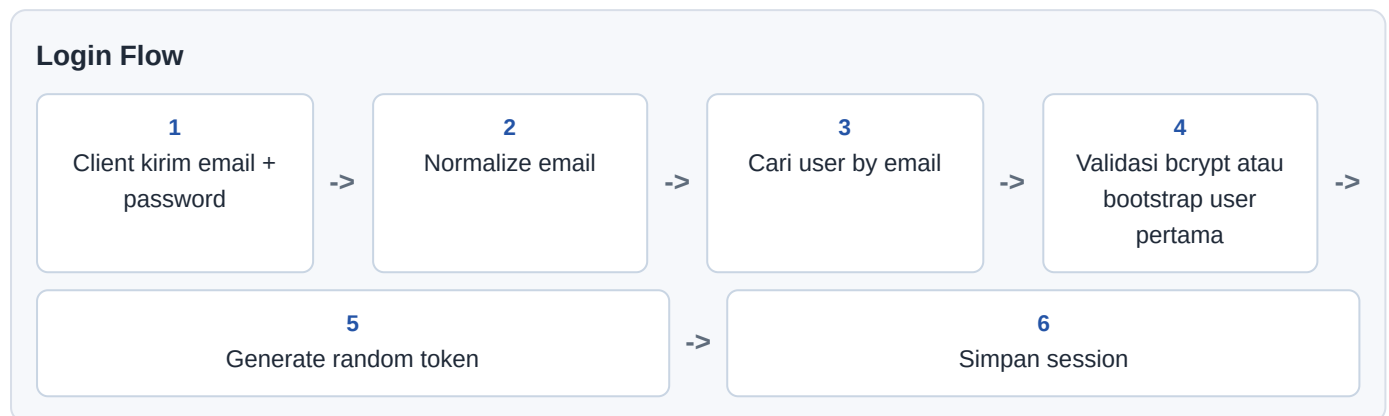
Endpoint Protected



Query session memakai kondisi `token = ? AND expired_at > now`. Jika token kosong, salah, atau sudah expired, API membalas `401 Unauthorized`.

5. Flow Autentikasi

Login



Endpoint login menerima JSON berisi `email` dan `password`. Email diubah menjadi lowercase dan spasi dihapus. Jika email atau password kosong, API membalas `401`.

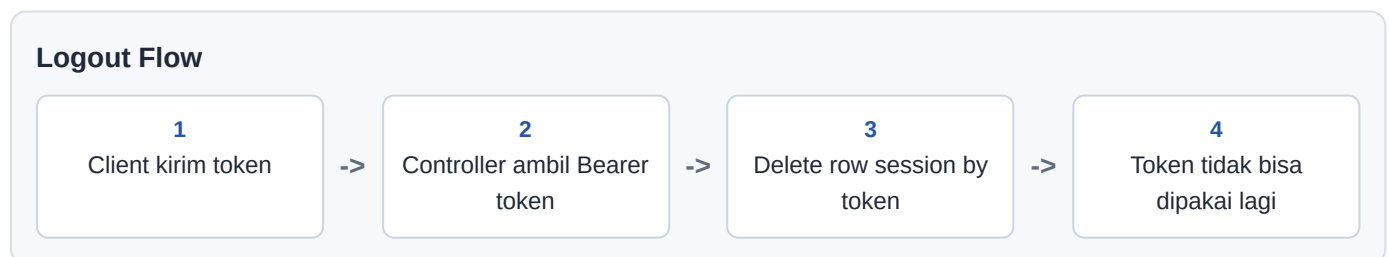
Jika user tidak ditemukan dan tabel `users` masih kosong, sistem otomatis membuat user pertama dengan role `SUPER_ADMIN`. Password disimpan memakai hash bcrypt. Ini adalah flow bootstrap admin awal. Jika tabel sudah berisi user, login email yang tidak ditemukan akan gagal.

Token dibuat dari 48 byte random, dikodekan dengan base64 URL-safe tanpa padding, lalu disimpan pada tabel `sessions` bersama `user_id`, `expired_at`, dan `created_at`. Masa berlaku token mengikuti `SESSION_TTL`, default `24h`.

Me

Endpoint `GET /api/auth/me` dan `GET /api/me` membaca user dari `c.Locals("user")` yang sudah diisi middleware auth, lalu mengembalikan data profil tanpa password.

Logout



6. Flow Otorisasi Role

Sistem memiliki tiga role: `SUPER_ADMIN` , `ADMIN` , dan `USER` . Role dicek setelah session token valid. Jika role user tidak termasuk daftar yang diizinkan, API membalas `403 Forbidden` .

Area	SUPER_ADMIN	ADMIN	USER
Login, logout, me	Ya	Ya	Ya
List dan detail resource protected	Ya	Ya	Ya
Create user	Ya	Ya	Tidak
Update user	Ya	Ya	Tidak
Delete user	Ya	Tidak	Tidak
Create, update, delete resource selain user	Ya	Ya	Tidak

7. Flow CRUD Resource

Hampir semua resource memakai `ResourceController[T]` generik. Controller ini menerima model GORM dan nama resource, lalu menyediakan operasi list, create, detail, update, dan delete.

Method	Path	Controller Flow	Akses
GET	<code>/api/<resource></code>	Ambil semua row dengan <code>Preload(clause.Associations)</code> .	Semua user login
POST	<code>/api/<resource></code>	Parse JSON body ke model, lalu <code>DB.Create</code> .	<code>SUPER_ADMIN</code> , <code>ADMIN</code>
GET	<code>/api/<resource>/:id</code>	Parse ID, ambil satu row dengan preload relasi.	Semua user login
PUT / PATCH	<code>/api/<resource>/:id</code>	Ambil row, parse body ke map, hapus field <code>id</code> dan <code>created_at</code> , update row.	<code>SUPER_ADMIN</code> , <code>ADMIN</code>
DELETE	<code>/api/<resource>/:id</code>	Ambil row, lalu <code>DB.Delete</code> .	<code>SUPER_ADMIN</code> , <code>ADMIN</code>

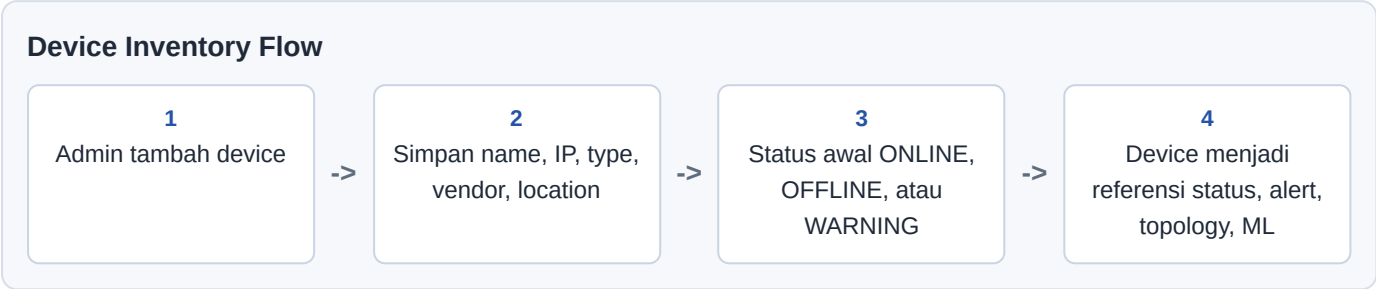
Flow CRUD User Khusus

User memakai controller sendiri karena perlu hashing password dan normalisasi field. Create user mewajibkan `name`, `email`, dan `password`. Jika role tidak dikirim, default role adalah `USER`. Update user hanya mengubah field yang dikirim dan password baru akan di-hash ulang.

8. Flow Domain Monitoring

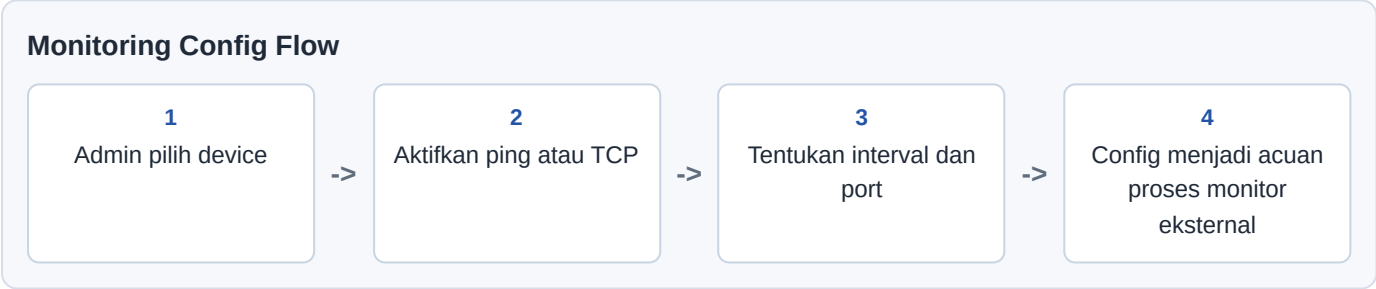
Bagian ini menjelaskan alur bisnis berdasarkan model dan endpoint yang tersedia. Implementasi saat ini adalah backend API penyimpanan data. Proses ping, TCP check, pengiriman notifikasi eksternal, atau inferensi ML dapat dilakukan oleh frontend, worker, scheduler, atau service lain yang mengonsumsi API ini.

8.1 Inventory Device



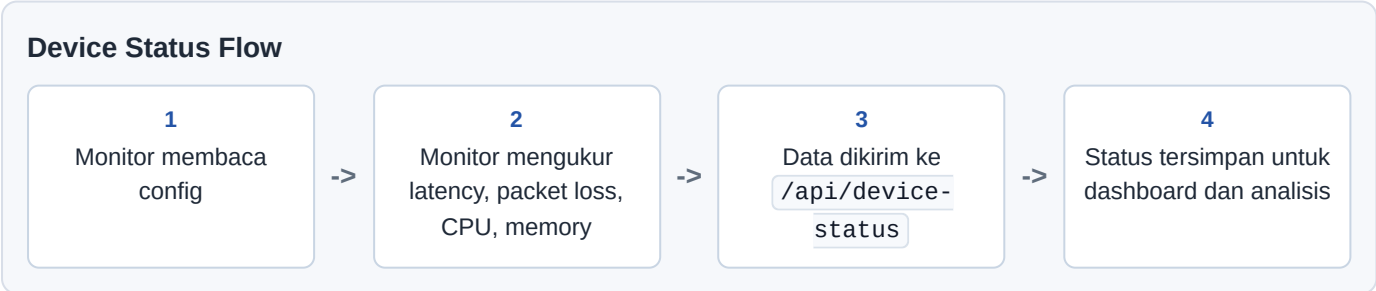
Device memiliki tipe `AP` atau `SERVICE`. Status device memakai enum `ONLINE`, `OFFLINE`, dan `WARNING`.

8.2 Monitoring Config



Konfigurasi menyimpan `ping_enabled`, `tcp_enabled`, `ping_interval`, `tcp_interval`, dan `monitored_port`. Default ping aktif, TCP tidak aktif.

8.3 Device Status



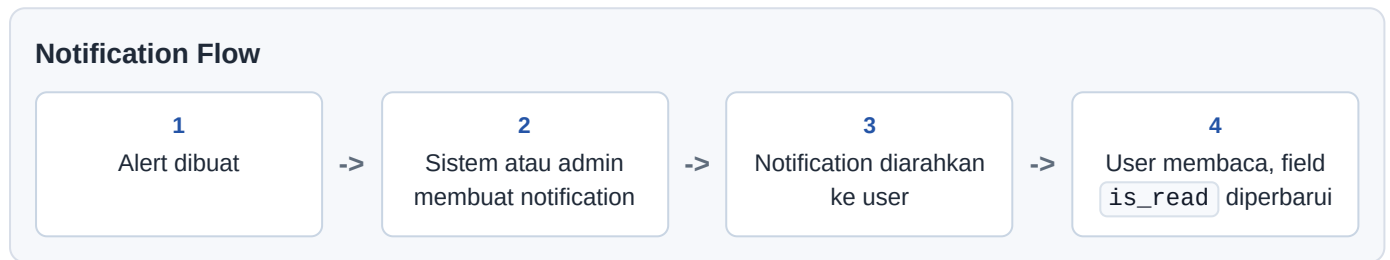
Tabel `device_status` menyimpan metrik numerik dan `last_seen`. Relasinya ke `devices` memakai `device_id`.

8.4 Alert



Alert menghubungkan masalah ke device tertentu, menyimpan pesan, severity, status, dan waktu pembuatan.

8.5 Notification

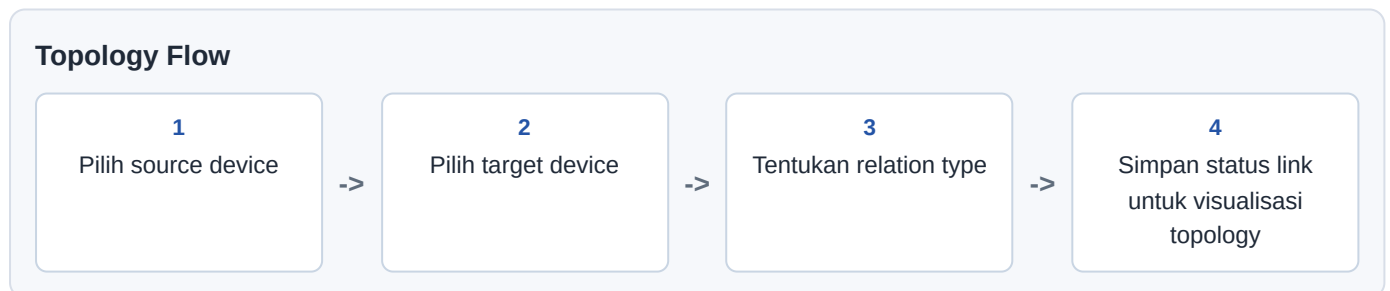


Notification menyimpan `user_id`, `alert_id`, judul, pesan, status baca, dan waktu dibuat.

8.6 Activity Log

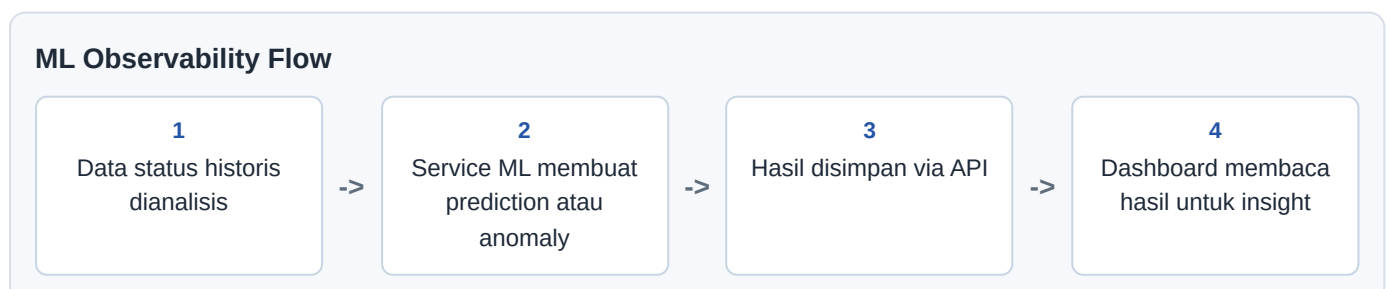
Activity log menyimpan jejak aktivitas user, misalnya `CREATE_DEVICE`, `UPDATE_ALERT`, atau aktivitas administratif lain. Model menyediakan field `user_id`, `action`, `description`, dan `created_at`. Pencatatan otomatis belum dipasang sebagai middleware, sehingga data log saat ini dibuat melalui endpoint CRUD.

8.7 Network Topology



Topology menyimpan hubungan antar dua device menggunakan `source_device_id` dan `target_device_id`. Contoh relation type adalah `uplink`, sedangkan status dapat berisi nilai seperti `active` atau `inactive`.

8.8 ML Prediction dan ML Anomaly



`ml_predictions` menyimpan tipe prediksi, nilai prediksi, dan confidence score. `ml_anomalies` menyimpan anomaly score, ringkasan prediksi, dan severity `WARNING` atau `CRITICAL`.

9. Flow Database dan Relasi Data

Semua model didaftarkan di `ModelsForMigration()` sehingga GORM membuat atau memperbarui tabel saat startup. Relasi memakai foreign key dengan constraint cascade update/delete pada model yang memiliki relasi eksplisit.

Tabel	Field Utama	Relasi
users	id, name, email, password, role, created_at	Menjadi parent untuk session, notification, activity log.
sessions	id, user_id, token, expired_at, created_at	user_id ke users.id.
devices	id, name, ip, type, vendor, location, status	Parent untuk status, config, alert, topology, prediction, anomaly.
device_status	device_id, latency, packet_loss, cpu_usage, memory_usage, last_seen	device_id ke devices.id.
monitoring_configs	device_id, ping_enabled, tcp_enabled, ping_interval, tcp_interval, monitored_port	device_id ke devices.id.
alerts	device_id, severity, message, status, created_at	device_id ke devices.id.
notifications	user_id, alert_id, title, message, is_read	user_id ke users.id, alert_id ke alerts.id.
activity_logs	user_id, action, description, created_at	user_id ke users.id.
network_topology	source_device_id, target_device_id, relation_type, status	Dua relasi ke devices.id.
ml_predictions	device_id, prediction_type, prediction_value, confidence_score	device_id ke devices.id.
ml_anomalies	device_id, anomaly_score, prediction, severity	device_id ke devices.id.

Saat controller list/detail menjalankan preload association, response dapat membawa object relasi seperti `device`, `user`, `alert`, `source_device`, atau `target_device` jika relasi tersedia.

10. Daftar Endpoint

Root dan Health

Method	Path	Akses	Fungsi
GET	/	Publik	Info API dan daftar endpoint.
GET	/healthz	Publik	Health check aplikasi.

Auth

Method	Path	Akses	Fungsi
POST	/api/auth/login , /api/login	Publik	Login dan membuat session token.
GET	/api/auth/me , /api/me	User login	Melihat profil user aktif.
POST	/api/auth/logout	User login	Hapus session token aktif.

Resource

Resource	Base Path	Operasi	Catatan Akses
Users	/api/users	List, create, detail, update, delete	Delete hanya SUPER_ADMIN .
Sessions	/api/sessions	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
Devices	/api/devices	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
Device Status	/api/device-status	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
Monitoring Configs	/api/monitoring-configs	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
Alerts	/api/alerts	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
Notifications	/api/notifications	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
Activity Logs	/api/activity-logs	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
Network Topology	/api/network-topology	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
ML Predictions	/api/ml-predictions	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .
ML Anomalies	/api/ml-anomalies	List, create, detail, update, delete	Mutasi oleh SUPER_ADMIN / ADMIN .

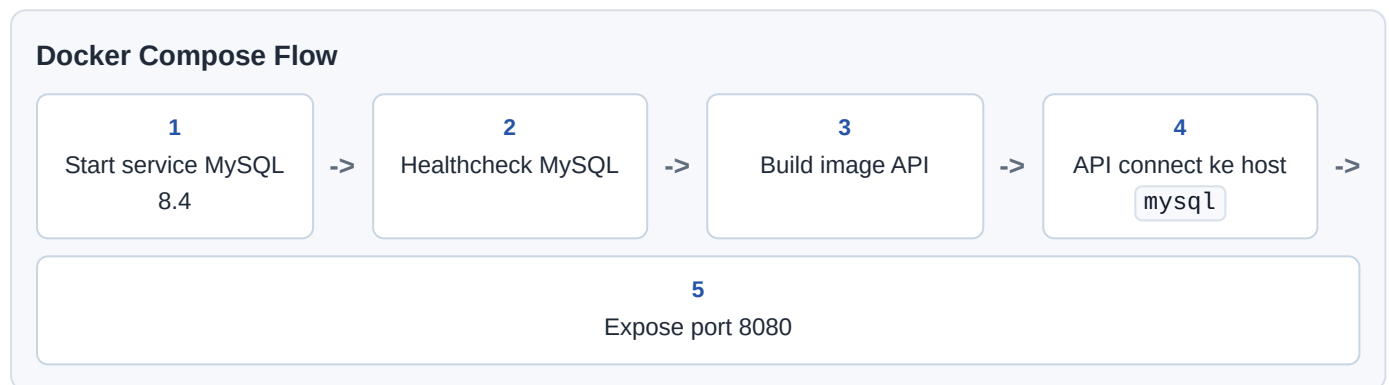
11. Flow Deployment

Local Development

```
cp .env.example .env
go run .
```

Secara default aplikasi berjalan di `http://localhost:8080`. Health check dapat dicek melalui `GET /healthz`.

Docker Compose



Dockerfile memakai multi-stage build: stage pertama memakai image Go Alpine untuk build binary `/out/netmonitor`, stage runtime memakai Alpine, membuat user non-root `appuser`, lalu menjalankan `/app/netmonitor`.

Catatan: `docker-compose.yml` mereferensikan file `./migrations/001_mysql_observability.sql`, tetapi folder/file tersebut tidak terlihat di working tree saat dokumen ini dibuat. Karena aplikasi sudah memakai GORM `AutoMigrate`, schema tetap dapat dibuat oleh backend, tetapi bind mount Docker Compose perlu dicek jika menjalankan compose.

12. Error Handling dan Response

Controller memakai format response JSON yang konsisten. Jika operasi berhasil, response biasanya memakai field `data` atau `message` . Jika gagal, response memakai field `error` dan kadang `detail` .

Kondisi	Status	Response
Body JSON tidak valid	400	<code>{"error":"invalid request body","detail":"..."}</code>
ID URL tidak valid	400	<code>{"error":"invalid id","detail":"..."}</code>
Login gagal	401	<code>{"error":"email or password is invalid"}</code>
Token kosong	401	<code>{"error":"authentication token is required"}</code>
Token salah atau expired	401	<code>{"error":"authentication token is invalid or expired"}</code>
Role tidak cukup	403	<code>{"error":"insufficient permission"}</code>
Data tidak ditemukan	404	<code>{"error":"resource not found"}</code>
Error database/server	500	<code>{"error":"failed to ...","detail":"..."}</code>

Contoh Flow API Singkat

```
POST /api/auth/login
Content-Type: application/json

{"email":"admin@example.com","password":"secret123"}

Response:
{
  "message": "login berhasil",
  "token": "SESSION_TOKEN",
  "session": {...},
  "user": {...}
}
```

```
GET /api/devices
Authorization: Bearer SESSION_TOKEN

Response:
{
  "data": []
}
```

13. Catatan Implementasi

- Auth saat ini memakai session token random yang disimpan di database, bukan JWT signed token.
- `JWT_SECRET` tetap ada di konfigurasi dan divalidasi untuk production, tetapi belum dipakai dalam flow auth.
- CRUD generik tidak memvalidasi aturan bisnis khusus per resource selain validasi constraint database dan parsing body.
- Monitoring aktif, scheduler ping/TCP, notifikasi eksternal, dan mesin ML belum terlihat sebagai service di repo ini.
- Activity log belum otomatis dibuat dari setiap aksi; endpoint CRUD tersedia untuk menyimpan log.
- File `README.md` dan `NetMonitor_API_Spec.md` tercatat terhapus di working tree saat dokumen ini dibuat; isi versi Git terakhir dipakai sebagai referensi tambahan.

Referensi File yang Dibaca

`routes/api.go`, `app/models/models.go`, `app/http/controllers/auth_controller.go`,
`app/http/controllers/user_controller.go`, `app/http/controllers/crud_controller.go`,
`app/http/middleware/auth.go`, `bootstrap/app.go`, `config/app.go`, `config/logger.go`,
`database/database.go`, `Dockerfile`, `docker-compose.yml`, `.env.example`,
`docs/observability_architecture.md`, dan versi Git terakhir dari `README.md` serta `NetMonitor_API_Spec.md`.